Resumen de Fundamentos de Programación en C++

Material de repaso — Ingeniería Biomédica · 2° semestre

> **Material de repaso - Ingeniería Biomédica, 2° semestre.** Este documento resume todo lo visto en el curso. Úsenlo como guía rápida en la víspera del trabajo final: cada tema va con su sintaxis, un ejemplo corto y las trampas más comunes.
>
> **Convención del curso:** trabajamos con **C++ puro** (`iostream`, `string`, `vector`), no con C clásico. Se evitan los **apuntadores**: cuando algo se pasa o se guarda "por referencia", usamos `&` (referencias) y también usamos `vector`.

1. Estructura básica de un programa

Todo programa en C++ tiene un punto de entrada: la función `main`. El compilador siempre empieza ahí.

```
#include <iostream>   // Entrada/salida: cin, cout
#include <vector>     // Contenedor dinámico de arreglos
#include <string>     // Cadenas de texto de C++
using namespace std;  // Evita escribir std:: en todo

int main() {
    cout << "Hola, ingeniería biomédica!" << endl;
    return 0;   // 0 = el programa terminó sin errores
}
```

Partes fijas que no deben faltar:

- `#include <...>`: librerías que se "pegan" antes de compilar.
- `using namespace std;`: para usar `cout`, `cin`, `string`, `vector` sin el prefijo `std::`.
- `int main() { ... }`: función obligatoria. Aquí empieza todo.
- Cada **instrucción (statement)** termina en **punto y coma** `;`.
- Los bloques de código se delimitan con llaves `{ }`.

Comentarios:

```
// Comentario de una línea
/* Comentario
   de varias líneas */
```

2. Tipos de datos

Una variable es un espacio en memoria con un **nombre**, un **tipo** y un **valor**. El tipo determina qué operaciones son válidas y cuánta memoria ocupa.

2.1 Tipos primitivos

Tipo	Qué guarda	Ejemplo	Tamaño típico
`int`	Enteros	`int pacientes = 30;`	4 bytes
`double`	Reales (decimales)	`double temperatura = 36.8;`	8 bytes
`float`	Reales (menos precisión)	`float ph = 7.4f;`	4 bytes
`char`	Un solo carácter	`char grupo = 'A';` (comillas simples)	1 byte
`bool`	Verdadero / falso	`bool activo = true;`	1 byte
`string`	Cadena de texto (C++)	`string nombre = "ECG";` (comillas dobles)	variable
`void`	"Nada" (usado en funciones)	`void mostrar();`	--

> **Nota:** `string` no es primitivo: es un tipo de la librería estándar. Pero lo usamos como si lo fuera. Para números con decimales **preferimos** `double` (más precisión que `float`, y evita errores de redondeo en cálculos de señales o dosis).

2.2 Constantes y modificadores

```
const double PI = 3.14159265;      // const: no puede cambiar nunca
double dosis = 0.5;
```

- `const`: su valor no puede modificarse después de inicializarse. Si lo intentas, **error de compilación** (eso es bueno: el compilador te protege).
- `unsigned` (mención breve): enteros solo positivos; el método `.size()` de `vector` devuelve uno.

2.3 Conversión de tipos (casting)

A veces necesitamos cambiar un tipo a otro. **Siempre usamos `static_cast`, nunca la forma de C.**

```
int muestras = 7;
double promedio = static_cast<double>(muestras) / 2;   // 3.5
```

Conversión implícita (automática): en `int + double`, C++ promueve el `int` a `double`.

Trampa clásica: dividir dos `int` trunca el resultado.

```
int a = 7, b = 2;
cout << a / b;                          // 3  (¡se pierde el decimal!)
cout << static_cast<double>(a) / b;  // 3.5
```

2.4 Inicialización

```
int x = 5;          // clásica
int y{5};           // moderna (llaves): no deja convertir tipos
incompatibles
double z{};         // inicializa en 0
```

3. Instrucciones y operadores

Un **statement** es una orden completa que el compilador ejecuta. Casi todos terminan en `;`. Hay de tres clases:

1. **Declaración** - crean variables: `int edad;`
2. **Asignación / expresión** - calculan: `promedio = suma / n;`
3. **Control** - deciden o repiten: `if`, `for`, `while`, `return`...

3.1 Asignación

```
fc = 72;            // guarda 72 en fc
fc = fc + 8;        // lee fc, suma 8 y lo vuelve a guardar (80)
fc += 8;            // forma corta (equivale a la línea anterior)
```

3.2 Operadores aritméticos

Operador	Significado	Ejemplo
`+ -`	Suma, resta	`a + b`
`* /`	Multiplicación, división	`a * b`
`%`	Módulo (residuo)	`17 % 5` da `2`
`++ --`	Incremento/decremento en 1	`i++`

> El módulo `%` solo funciona con enteros y es muy útil: `if (n % 2 == 0)` significa número par. `if (m % 60 == 0)` significa cada minuto exacto de una señal muestreada.

3.3 Operadores relacionales y lógicos

Devuelven `bool` (`true` o `false`):

Operador	Significado	Operador	Significado
`==`	igual a	`!=`	diferente de
`<`	menor que	`<=`	menor o igual
`>`	mayor que	`>=`	mayor o igual
`&&`	Y lógico	`	
`!`	NO lógico		

```
if (edad >= 18 && edad < 65) { ... }   // ambos deben cumplirse
if (genero == 'M' || genero == 'F') { ... }
```

> **Trampa #1 del semestre:** `=` **asigna**, `==` **compara**. Escribir `if (x = 5)` compila y hace lo incorrecto.

Precedencia sugerida: ante la duda, usen paréntesis. `a + b * c` no es lo mismo que `(a + b) * c`, y `(a && b) || c` no es lo mismo que `a && (b || c)`.

3.4 Entrada y salida

```
cout << "Digite la edad: ";          // imprime sin salto
cin  >> edad;                         // lee del teclado
cout << "Edad: " << edad << endl;     // endl = salto de línea
```

Formato de decimales (necesario al imprimir promedios o señales):

```
#include <iomanip>
cout << fixed << setprecision(2);   // a partir de aquí: 2 decimales
cout << 36.789;                     // 36.79
```

Leer texto con espacios: `getline(cin, nombre);` (usar después de `cin >>`, anteponiendo `cin.ignore()`).

4. Estructuras de control (decisión)

Permiten que el programa **tome caminos distintos** según una condición.

4.1 `if` / `else` / `else if`

```
double fc;
cin >> fc;

if (fc < 60) {
    cout << "Bradicardia";
```

```
} else if (fc >= 60 && fc <= 100) {
    cout << "Frecuencia normal";
} else {
    cout << "Taquicardia";
}
```

- El bloque `else` es opcional; `else if` se puede encadenar.
- Las condiciones se evalúan **en orden**: la primera que sea verdadera gana y el resto se ignora.
- Si el cuerpo tiene una sola línea se pueden omitir las llaves, pero **en este curso exigimos llaves siempre** (evita errores al agregar líneas).

4.2 `switch`

Útil cuando una variable se compara contra varios valores **exactos** (típicamente `int` o `char`).

```
char opcion;
cin >> opcion;

switch (opcion) {
    case '1':
        cout << "Calcular IMC";
        break;              // sin break, "cae" al siguiente caso ¡error típico!
    case '2':
        cout << "Ver pacientes";
        break;
    default:
        cout << "Opción inválida";
}
```

> `break` sale del `switch`. `default` se ejecuta si ningún caso coincide (es el "else" del switch).

4.3 Operador ternario

Forma corta de un `if/else` que solo asigna o devuelve un valor:

```
string estado = (spo2 >= 95) ? "Adecuada" : "Hipoxemia";
```

Sintaxis: `condicion ? valorSiVerdadero : valorSiFalso`

5. Estructuras de repetición (ciclos)

Permiten **repetir instrucciones** sin reescribirlas. Toda repetición tiene 3 partes: **inicialización, condición y actualización**.

5.1 `while` - "mientras"

```
int n = 1;
while (n <= 5) {        // condición revisada ANTES de cada vuelta
    cout << n << " ";
    n++;                // ¡sin esto, el ciclo es infinito!
}
```

Se usa cuando **no se sabe de antemano** cuántas veces se repetirá (ej.: validar entrada hasta que el usuario digite algo correcto).

5.2 `do-while` - "hacer... mientras"

```
double presion;
do {
    cout << "Digite presión arterial (> 0): ";
    cin >> presion;
} while (presion <= 0);   // condición revisada DESPUÉS: el cuerpo corre al menos 1 vez
```

Ideal para **validaciones**: primero pregunta, luego revisa.

5.3 `for` - ciclo contable

```
for (int i = 0; i < 10; i++) {   // init; condición; actualización
    cout << "Muestra " << i << endl;
}
```

Se usa cuando **se sabe cuántas vueltas** se necesitan. Convención: los índices empiezan en **0**.

5.4 `for` por rango (range-based)

La forma moderna y más limpia de **recorrer** un contenedor completo:

```
vector<double> senal = {0.1, 0.5, 1.2, 0.8};
for (double v : senal) {      // "para cada valor v dentro de senal"
    cout << v << " ";
}
```

> Para **modificar** los elementos dentro del ciclo, se recibe por referencia: `for (double& v : senal) { v = v * 2; }`

5.5 `break` y `continue`

```cpp
for (double m : senal) {
    if (m < 0) continue;    // salta esta vuelta y sigue con la siguiente
    if (m > 5.0) break;     // abandona el ciclo por completo
    cout << m << " ";
}
```

Resumen de ciclos:

Ciclo	Cuándo usarlo	Revisa antes o después	Vueltas garantizadas
`while`	No se sabe cuántas veces	Antes	0 o más
`do-while`	Validaciones (mínimo un intento)	Después	1 o más
`for`	Número de vueltas conocido	Antes	0 o más

6. Funciones

Una **función** es un bloque de código con nombre que hace una tarea y puede **reutilizarse**. Es la herramienta principal para dividir un problema grande (como su trabajo final) en partes pequeñas y probables.

6.1 Anatomía

```cpp
//   tipo   nombre(parámetros)
double imc(double peso, double altura) {
    return peso / (altura * altura);   // devuelve un double
}
```

- **Tipo de retorno:** qué devuelve (`double`, `int`, `bool`, `string`... o `void` si no devuelve nada).
- **Parámetros:** los datos que recibe, cada uno con su tipo.
- **`return`:** devuelve el valor y termina la función. En `void` se puede usar `return;` solo para salir.

6.2 Prototipo: declarar antes, definir después

El compilador lee de arriba hacia abajo. Si `main` llama a una función definida más abajo, hay que **declarar el prototipo** arriba:

```cpp
double imc(double peso, double altura);   // prototipo: termina en ;

int main() {
    cout << imc(70, 1.75);                // ya puede usarse
    return 0;
}

double imc(double peso, double altura) { // definición completa
    return peso / (altura * altura);
}
```

6.3 Paso de parámetros: valor vs. referencia

Esto reemplaza lo que en C se hace con apuntadores:

```cpp
// POR VALOR: la función recibe una COPIA. El original no cambia.
void duplicarCopia(int x) { x = x * 2; }

// POR REFERENCIA (&): la función recibe el ORIGINAL. Sí lo cambia.
void duplicar(int& x)    { x = x * 2; }

// REFERENCIA CONSTANTE: recibe el original pero solo para leerlo.
// Es la forma correcta de pasar strings, vectores o structs grandes.
void mostrar(const string& nombre) { cout << nombre; }
```

```cpp
int a = 5;
duplicarCopia(a);  cout << a;  // 5 (no cambió)
duplicar(a);       cout << a;  // 10 (¡cambió!)
```

Regla práctica del curso:

- Tipos pequeños (`int`, `double`, `char`, `bool`): **por valor**.
- Quieres que la función **modifique** lo que recibe: **referencia** `&`.
- Solo quieres **leer** algo grande (`vector`, `string`, `struct`): **`const&`**.

6.4 Sobrecarga

Varias funciones pueden llamarse **igual** si sus parámetros son distintos (en tipo o cantidad):

```cpp
double promedio(double a, double b);
double promedio(double a, double b, double c);
double promedio(const vector<double>& v);
```

C++ elige la versión correcta según lo que se le envíe. Mismo nombre, comportamientos a la medida.

6.5 Alcance de variables (scope)

```
int global = 10;          // visible en todo el archivo (usar con
moderación)

void f() {
    int local = 5;        // solo existe dentro de f()
}                         // aquí local "muere"
```

Una variable declarada dentro de un bloque `{ }` (función, `if`, ciclo) **solo existe dentro de ese bloque**. Declarar `int i` dentro del `for` no choca con otro `i` externo.

6.6 Funciones y estructuras de datos

Las funciones son el "motor" de todo lo que sigue: se puede pasar y devolver `vector`, matrices y `struct` (con las reglas de `&` y `const&` de arriba).

```
double promedio(const vector<double>& v) {
    double suma = 0;
    for (double x : v) suma += x;
    return suma / v.size();
}
```

7. Arreglos (colecciones de datos)

Un **arreglo** guarda muchos valores del mismo tipo bajo un solo nombre, cada uno con una **posición (índice)** que empieza en **0**.

7.1 Arreglos tradicionales (vista rápida, para lectura)

```
double senal[5] = {0.2, 0.4, 0.9, 1.1, 0.7};  // tamaño FIJO: 5, no puede
crecer
cout << senal[0];     // 0.2 (primer elemento)
cout << senal[4];     // 0.7 (último: tamaño - 1)
```

Los vimos para entender la base histórica, pero tienen limitaciones serias: el tamaño debe conocerse de antemano, **no saben cuántos elementos tienen** y no verifican índices. Por eso, en este curso, **casi siempre usamos `vector`**.

7.2 `vector` - el arreglo de C++ (nuestra herramienta principal)

```
#include <vector>

vector<double> fc;                          // vacío
vector<double> senal = {0.2, 0.4, 0.9};     // inicializado
```

```
vector<int> conteo(10);                    // 10 enteros, todos en 0
vector<double> buffer(100, 0.0);           // 100 doubles, todos en 0.0
```

Métodos esenciales:

Método / operador	Qué hace
`v.push_back(x)`	Agrega `x` **al final**
`v.size()`	Cuántos elementos hay
`v[i]`	Accede a la posición `i` (sin verificación)
`v.at(i)`	Accede a la posición `i` (**verifica** y lanza error si `i` es inválido)
`v.pop_back()`	Elimina el último
`v.clear()`	Vacía el vector
`v.empty()`	`true` si no tiene elementos

```
vector<double> frecuencias;
double f;
for (int i = 0; i < 5; i++) {
    cin >> f;
    frecuencias.push_back(f);     // crece solo: no hay que manejar memoria
}

cout << "Total: " << frecuencias.size() << endl;

for (double f : frecuencias)      // recorrerlo: range-based for
    cout << f << endl;
```

> **Ojo:** siempre que usen `v[i]`, verifiquen que `i < v.size()`. Salirse del rango **no siempre da error visible**: puede devolver basura o congelar el programa. `v.at(i)` es la opción segura cuando dudan.

8. Matrices (arreglos de dos dimensiones)

Una matriz organiza datos en **filas y columnas**: una imagen médica es una matriz de píxeles; una tabla de pacientes vs. variables es una matriz.

8.1 Matrices "normales": `vector< vector<T> >`

Un vector **cuyos elementos son vectores** (vectores anidados):

```
// matriz de 3 filas x 4 columnas, inicializada en 0
vector<vector<double>> imagen(3, vector<double>(4, 0.0));

// escribir: [fila][columna]
imagen[1][2] = 128.0;
```

```
// leer
double px = imagen[1][2];

// recorrer: dos ciclos anidados
for (int i = 0; i < (int)imagen.size(); i++) {          // i = fila
    for (int j = 0; j < (int)imagen[i].size(); j++) {   // j = columna
        cout << imagen[i][j] << "\t";
    }
    cout << endl;   // salto de línea al terminar cada fila
}
```

Regla de oro: el primer índice es la **fila** y el segundo la **columna**. `m[i][j]` = fila `i`, columna `j`.

```
// matriz con valores dados:
vector<vector<int>> tabla = {
    {1, 2, 3},
    {4, 5, 6}      // 2 filas x 3 columnas
};
```

También se puede pasar a funciones (por referencia si se modifica, `const&` si solo se lee):

```
double promedio(const vector<vector<double>>& m) {
    double suma = 0; int n = 0;
    for (const auto& fila : m)
        for (double x : fila) { suma += x; n++; }
    return suma / n;
}
```

8.2 Matriz guardada como **fila mayor** (row-major)

Es la forma en que C++ **realmente** guarda una matriz en memoria: los bytes no forman una cuadrícula, son **una sola línea**. La memoria es lineal, así que la fila 0 va primero, luego la fila 1 completa, y así sucesivamente. Eso se llama **orden fila mayor**.

La matriz

```
[ 10  20  30 ]
[ 40  50  60 ]
```

se guarda en memoria, fila por fila, como:

```
posición:  0    1    2    3    4    5
valor:    10   20   30   40   50   60
```

```
            [---- fila 0 ----][- fila 1 ---]
```

Por eso podemos **guardar una matriz dentro de un vector de una sola dimensión**, calculando la posición con una fórmula:

```
indice = fila * NUM_COLUMNAS + columna
```

```cpp
// Matriz 2x3 "aplanada" en un vector (fila mayor)
int filas = 2, cols = 3;
vector<double> m(filas * cols, 0.0);

// Escribir el valor de la fila 1, columna 2:
m[1 * cols + 2] = 60.0;      // = m[5]

// Recorrer toda la matriz:
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < cols; j++) {
        cout << m[i * cols + j] << " ";
    }
    cout << endl;
}
```

El truco inverso (de posición lineal a fila/columna), útil cuando se recorre un `vector` y se quiere saber en qué fila cae:

```
fila    = indice / cols
columna = indice % cols
```

¿Por qué importa?

Aspecto	`vector<vector<T>>`	Fila mayor (vector aplanado)
Legibilidad	Más natural, se ve como matriz	Hay que calcular índices
Memoria	Filas dispersas por memoria	**Contigua**: una sola línea
Velocidad al recorrer en orden	Buena	**Mejor** (la CPU lee seguido)
Cuándo usarla	Código general, tablas medianas	Imágenes, señales grandes, cuando importa el rendimiento o se reciben datos aplanados de un sensor o archivo

En instrumentación y procesamiento de imágenes es común recibir los datos **ya aplanados** (por ejemplo, un frame de un sensor llega como una cadena de `f*c` muestras): fila mayor es exactamente el convenio que se usa para reinterpretarlos como matriz.

9. Structs: agrupando datos con comportamiento

Un `struct` permite crear **nuestro propio tipo de dato** que agrupa variables relacionadas (**atributos**) y también puede tener **funciones propias** (**métodos**). Es la herramienta natural para modelar entidades del mundo biomédico.

> En este curso usamos structs **sin** los conceptos de programación orientada a objetos (herencia, polimorfismo, encapsulamiento, etc.): solo atributos y métodos. Un struct es "un paquete de datos con sus funciones".

9.1 Definir y usar

```
struct Paciente {
    // Atributos (las variables internas del struct)
    string nombre;
    int edad;
    double peso;
    double altura;

    // Métodos (las funciones internas del struct)
    double imc() {
        return peso / (altura * altura);
    }

    bool esAdultoMayor() {
        return edad >= 60;
    }

    void mostrar() {
        cout << nombre << " | " << edad << " años | IMC: "
             << imc() << endl;       // un método puede llamar a otro
    }
};
```

Importante: definir el struct solo crea el "molde". Para usarlo hay que declarar variables de ese tipo:

```
Paciente p1;                    // creación
p1.nombre = "Ana";               // acceso a atributos con el punto .
p1.edad = 34;
p1.peso = 62.0;
p1.altura = 1.68;

p1.mostrar();                   // llamada al método: también con punto
cout << p1.imc();               // 21.97

Paciente p2 = {"Luis", 70, 80.0, 1.75};   // inicialización en orden
```

9.2 Structs + arreglos: el patrón más útil del trabajo final

La combinación `vector<Paciente>` es la forma natural de manejar muchos registros:

```
vector<Paciente> pacientes;
Paciente p;
for (int i = 0; i < 3; i++) {
    cout << "Nombre: ";  cin >> p.nombre;
    cout << "Edad: ";     cin >> p.edad;
    cin >> p.peso >> p.altura;
    pacientes.push_back(p);
}

// Recorrer y usar métodos:
double sumaImc = 0;
for (const Paciente& q : pacientes) {   // const&: solo lectura, eficiente
    q.mostrar();
    sumaImc += q.imc();
}
cout << "IMC promedio: " << sumaImc / pacientes.size();
```

9.3 Structs y funciones

Los structs se pasan igual que cualquier dato, con la **regla de `const&`**: para leer, `const Paciente&`; para modificar, `Paciente&`.

```
void normalizar(Paciente& p) {            // modifica el original
    if (p.peso <= 0) p.peso = 1.0;
}

int contarAdultosMayores(const vector<Paciente>& lista) {  // solo lectura
    int conteo = 0;
    for (const Paciente& q : lista)
        if (q.esAdultoMayor()) conteo++;
    return conteo;
}
```

9.4 (Opcional) Constructor abreviado

Un struct puede tener una función especial que inicializa sus atributos al crearlo:

```
struct Muestra {
    double valor;
    double tiempo;
    Muestra(double v, double t) : valor(v), tiempo(t) {}
};

vector<Muestra> senal;
senal.push_back(Muestra(0.92, 0.005));   // se crea ya inicializada
```

No es obligatorio, pero ahorra líneas cuando se crean muchos registros.

10. Errores comunes (lista de verificación antes de entregar)

- `=` es asignación; `==` es comparación. Revisar todos los `if`.
- `cout` imprime, `cin` lee. No mezclar.
- Ciclos: comprobar que la variable de control **se actualiza**. Si no, ciclo infinito.
- Índices empiezan en **0**: el último de un `vector` de `n` es `n-1`.
- Dividir dos `int` trunca. Usar `static_cast<double>` para promedios.
- `switch`: cada `case` lleva su `break`.
- Funciones: si modificas un parámetro y no pasa nada, te faltó el `&`.
- `vector`: no usar `push_back` dentro de un ciclo basado en `v.size()` sin cuidado (el tamaño cambia debajo de ti).
- Punto y coma al final de cada statement... pero **nunca** después de `if (...)` ni de la cabecera de `for`/`while`.
- Compilar **a cada rato**, no al final: un error a tiempo vale por diez.
- Probar con casos límite: 0 pacientes, 1 muestra, valores negativos.

11. Tabla de repaso exprés

Tema	Palabra clave	Idea en una línea
Tipo de dato	`int`, `double`, `char`, `bool`, `string`	Determina qué guarda y qué operaciones permite
Conversión	`static_cast<double>(x)`	Cambia de tipo de forma segura
Decisión	`if`, `else`, `switch`, `?:`	El programa elige un camino según la condición
Repetición	`while`, `do-while`, `for`	Repite código controladamente
Recorrer	`for (T x : v)`	La forma corta de visitar cada elemento
Función	`tipo nombre(parámetros)`	Bloque reutilizable con entrada y salida
Paso por referencia	`&`, `const&`	Modifica el original / lee sin copiar
Arreglo	`vector<T>`	Muchos valores del mismo tipo, tamaño flexible
Matriz	`vector<vector<T>>`	Tabla de filas y columnas
Fila mayor	`v[i*cols + j]`	Matriz "aplanada" en una sola línea de memoria
Struct	`struct`, atributos, métodos	Tipo propio que agrupa datos y sus funciones

---

Creado como material de repaso del curso. Si algo no cuadra con lo visto en clase, la documentación y el compilador siempre tienen la última palabra. Éxitos en el trabajo final.